



Problem Challenge 1

We'll cover the following



- Permutation in a String (hard)
- Try it yourself

Permutation in a String (hard)

Given a string and a pattern, find out if the **string contains any permutation of the pattern**.

Permutation is defined as the re-arranging of the characters of the string. For example, "abc" has the following six



1. abc
2. acb
3. bac
4. bca
5. cab
6. cba

If a string has 'n' distinct characters, it will have $n!$ permutations.

Example 1:

Input: String="oidbcaf", Pattern="abc"

Output: true

Explanation: The string contains "bca" which is a permutation of the given pattern.

Example 2:

Input: String="odicf", Pattern="dc"

Output: false

Explanation: No permutation of the pattern is present in the given string as a substring.

Example 3:

Input: String="bcdxabc dy", Pattern="bcdyabcdx"

Output: true

Explanation: Both the string and the pattern are a permutation of each other.



Example 4:



Input: String="aaacb", Pattern="abc"

Output: true

Explanation: The string contains "acb" which is a permutation of the given pattern.



Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1 import java.util.*;
2
3 class StringPermutation {
4     public static boolean findPermutation(String input, String val) {
5         List<Character> arr = new ArrayList<>(3);
6         int c;
7         for(int end =0 ; end< input.length();end++){
8             char ch = input.charAt(end);
9             arr.add(ch);
10            if(arr.size() == val.length()){
11                c=0;
12                for(char ch1 : val.toCharArray()){
13                    if(!arr.contains(ch1))
14                        break;
15                    c++;
16                }
17                if(c == val.length()) return true;
18                arr.remove(0);
19            }
20        }
21
22        return false;
23    }
24 }
25
```



Test

Save

Reset



Back

Completed

Next

Longest Subarray with Ones after Replacement (hard)

Solution Review: Problem Challenge 1

